

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A **EXPERIMENTS**

Code of Conduct:

I Athavan k(192524036)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan k

EXPERIMENTS

Q1. You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

- (i) Accept an arithmetic expression with operators +, -, *, / and parentheses
- (ii) Use syntax-directed evaluation to compute values based on operator precedence
- (iii) Optimize evaluation by reducing unnecessary temporary computations
- (iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

$E \rightarrow E + T \quad \{ E.val = E1.val + T.val \}$
 $E \rightarrow E - T \quad \{ E.val = E1.val - T.val \}$ $E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow T * F \quad \{ T.val = T1.val * F.val \}$
 $T \rightarrow T / F \quad \{ T.val = T1.val / F.val \}$
 $T \rightarrow F \quad \{ T.val = F.val \}$

$F \rightarrow (E) \quad \{ F.val = E.val \}$
 $F \rightarrow \text{digit} \quad \{ F.val = \text{digit} \}$

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation. Write a C program that:

- (i) Evaluates a postfix expression
- (ii) Demonstrates S-attributed definition evaluation
- (iii) Shows how intermediate results are computed

Q3. You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child. Write a C program that:

- (i). Evaluates a simple expression like $a + b * c$
 - (ii) Uses function parameters to simulate inherited attributes
 - (iii). Ensures correct precedence handling
 - (iv). Demonstrates top-down evaluation logic
- Explain how inherited attributes influence computation.

Q4. In a compiler, it is important to check whether two type expressions are equivalent. Write a C program that:

- (i). Accepts two type expressions (e.g., int, float, int*)

- (ii).Compares them for **type equivalence**
 - (iii).Displays whether they are:
 - (a).Equivalent
 - (b).Not equivalent
- Extend the logic to handle pointer types (`int*`, `float*`).

Q5.You are building a semantic analyzer that detects type errors in arithmetic expressions.

Write a C program that:

- (i). Accepts two operands and their types
- (ii). Checks whether the operation (e.g., +, *) is valid
- (iii). Reports:
 - (a).Valid expression
 - (b).Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

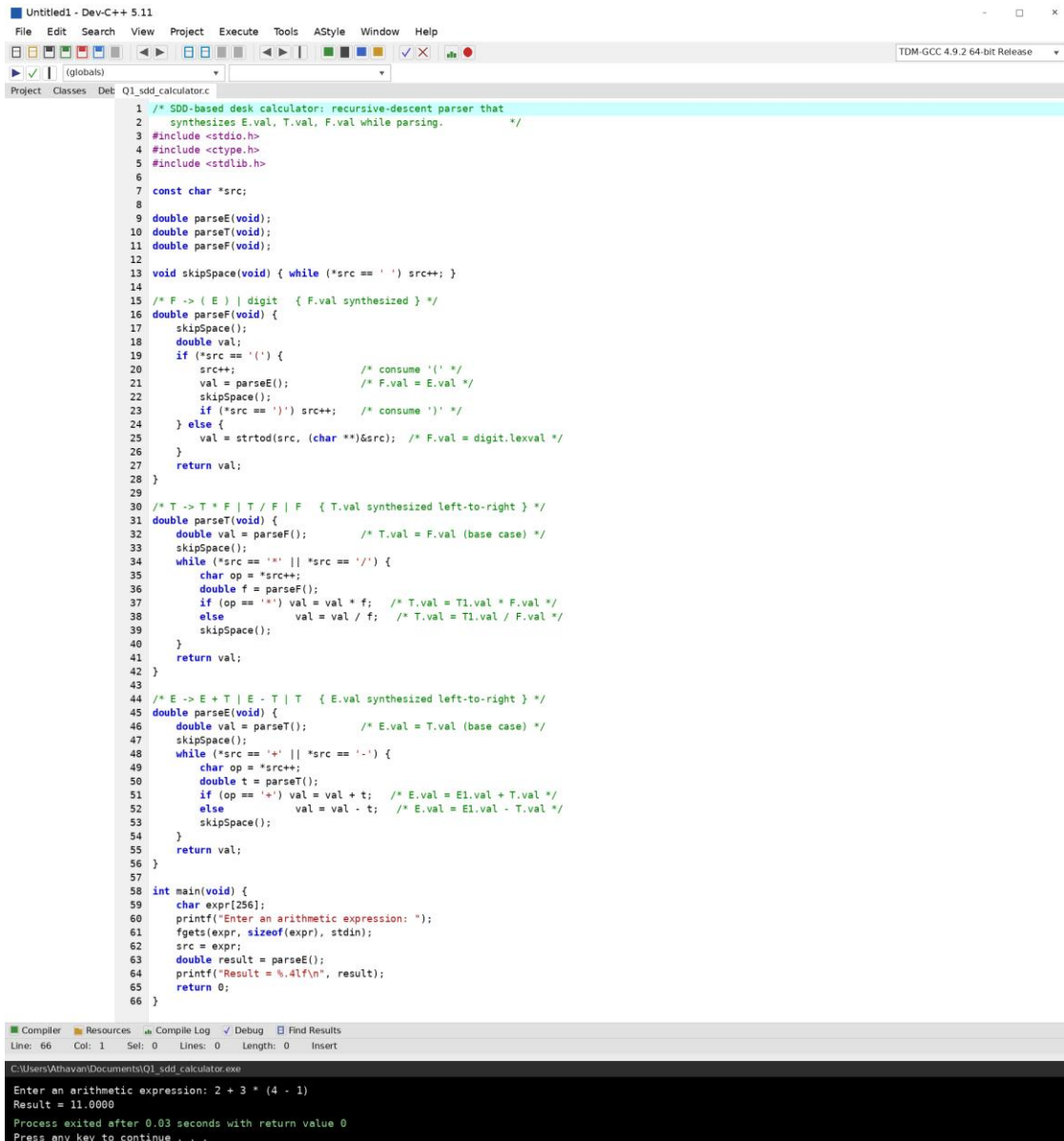
RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. Syntax-Directed Definition (SDD) for a Desk Calculator

- Accepts an arithmetic expression with +, -, *, / and parentheses.
- Uses synthesized attributes (E.val, T.val, F.val) to evaluate the expression honouring operator precedence.
- Evaluates each production's action immediately on reduction, avoiding unnecessary temporaries.

Program (Dev-C++) and Console Output:



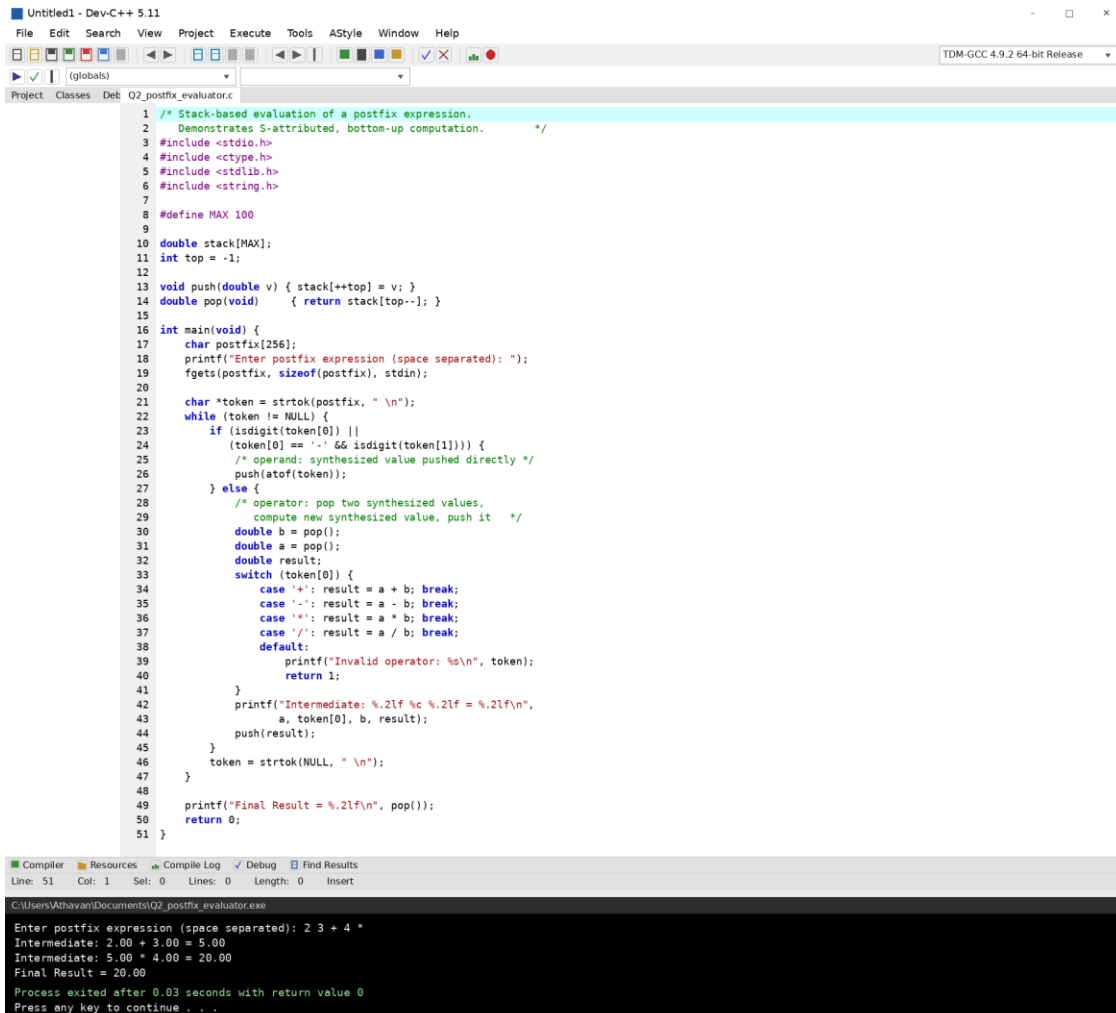
```
1  /* SDD-based desk calculator: recursive-descent parser that
2     synthesizes E.val, T.val, F.val while parsing. */
3  #include <stdio.h>
4  #include <ctype.h>
5  #include <stdlib.h>
6
7  const char *src;
8
9  double parseE(void);
10 double parseT(void);
11 double parseF(void);
12
13 void skipSpace(void) { while (*src == ' ') src++; }
14
15 /* F -> ( E ) | digit { F.val synthesized } */
16 double parseF(void) {
17     skipSpace();
18     double val;
19     if (*src == '(') {
20         src++; /* consume '(' */
21         val = parseE(); /* F.val = E.val */
22         skipSpace();
23         if (*src == ')') src++; /* consume ')' */
24     } else {
25         val = strtod(src, (char **)&src); /* F.val = digit.lexval */
26     }
27     return val;
28 }
29
30 /* T -> T * F | T / F | F { T.val synthesized left-to-right } */
31 double parseT(void) {
32     double val = parseF(); /* T.val = F.val (base case) */
33     skipSpace();
34     while (*src == '*' || *src == '/') {
35         char op = *src++;
36         double f = parseF();
37         if (op == '*') val = val * f; /* T.val = T1.val * F.val */
38         else val = val / f; /* T.val = T1.val / F.val */
39         skipSpace();
40     }
41     return val;
42 }
43
44 /* E -> E + T | E - T | T { E.val synthesized left-to-right } */
45 double parseE(void) {
46     double val = parseT(); /* E.val = T.val (base case) */
47     skipSpace();
48     while (*src == '+' || *src == '-') {
49         char op = *src++;
50         double t = parseT();
51         if (op == '+') val = val + t; /* E.val = E1.val + T.val */
52         else val = val - t; /* E.val = E1.val - T.val */
53         skipSpace();
54     }
55     return val;
56 }
57
58 int main(void) {
59     char expr[256];
60     printf("Enter an arithmetic expression: ");
61     fgets(expr, sizeof(expr), stdin);
62     src = expr;
63     double result = parseE();
64     printf("Result = %.4lf\n", result);
65     return 0;
66 }
```

Compiler Resources Compile Log Debug Find Results
Line: 66 Col: 1 Sel: 0 Lines: 0 Length: 0 Insert
C:\Users\Athavan\Documents\Q1_sdd_calculator.exe
Enter an arithmetic expression: 2 + 3 * (4 - 1)
Result = 11.0000
Process exited after 0.03 seconds with return value 0
Press any key to continue . . .

Q2. Postfix Expression Evaluation (S-Attributed, Stack-Based Bottom-Up)

- Evaluates a postfix expression using a stack.
- Demonstrates S-attributed definition evaluation — every attribute is synthesized from children already reduced on the stack.
- Prints each intermediate result as it is computed.

Program (Dev-C++) and Console Output:



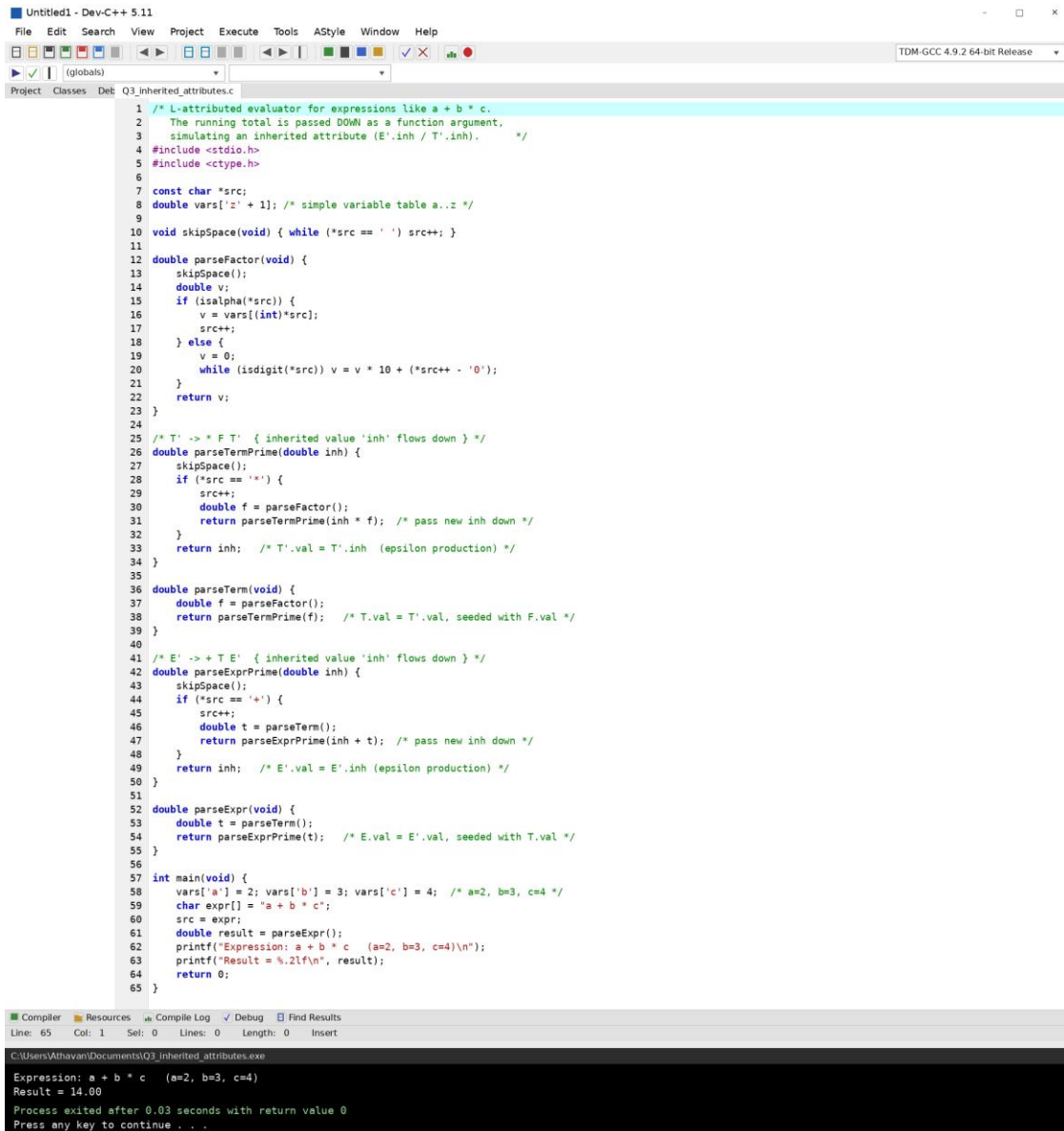
```
1  /* Stack-based evaluation of a postfix expression.
2  Demonstrates S-attributed, bottom-up computation. */
3  #include <stdio.h>
4  #include <ctype.h>
5  #include <stdlib.h>
6  #include <string.h>
7
8  #define MAX 100
9
10 double stack[MAX];
11 int top = -1;
12
13 void push(double v) { stack[++top] = v; }
14 double pop(void) { return stack[top--]; }
15
16 int main(void) {
17     char postfix[256];
18     printf("Enter postfix expression (space separated): ");
19     fgets(postfix, sizeof(postfix), stdin);
20
21     char *token = strtok(postfix, " \n");
22     while (token != NULL) {
23         if (isdigit(token[0]) ||
24             (token[0] == '-' && isdigit(token[1]))) {
25             /* operand: synthesized value pushed directly */
26             push(atof(token));
27         } else {
28             /* operator: pop two synthesized values,
29              compute new synthesized value, push it */
30             double b = pop();
31             double a = pop();
32             double result;
33             switch (token[0]) {
34                 case '+': result = a + b; break;
35                 case '-': result = a - b; break;
36                 case '*': result = a * b; break;
37                 case '/': result = a / b; break;
38                 default:
39                     printf("Invalid operator: %s\n", token);
40                     return 1;
41             }
42             printf("Intermediate: %.2lf %c %.2lf = %.2lf\n",
43                 a, token[0], b, result);
44             push(result);
45         }
46         token = strtok(NULL, " \n");
47     }
48     printf("Final Result = %.2lf\n", pop());
49     return 0;
50 }
51 }
```

Compiler Resources Compile Log Debug Find Results
Line: 51 Col: 1 Sel: 0 Lines: 0 Length: 0 Insert
C:\Users\Athavan\Documents\Q2_postfix_evaluator.exe
Enter postfix expression (space separated): 2 3 + 4 *
Intermediate: 2.00 + 3.00 = 5.00
Intermediate: 5.00 * 4.00 = 20.00
Final Result = 20.00
Process exited after 0.03 seconds with return value 0
Press any key to continue . . .

Q3. L-Attributed Definition — Inherited Attributes (Top-Down Evaluation)

- Evaluates an expression like $a + b * c$.
- Uses function parameters to simulate inherited attributes (the running total is passed down the call chain).
- Ensures correct precedence handling and demonstrates top-down evaluation logic.

Program (Dev-C++) and Console Output:



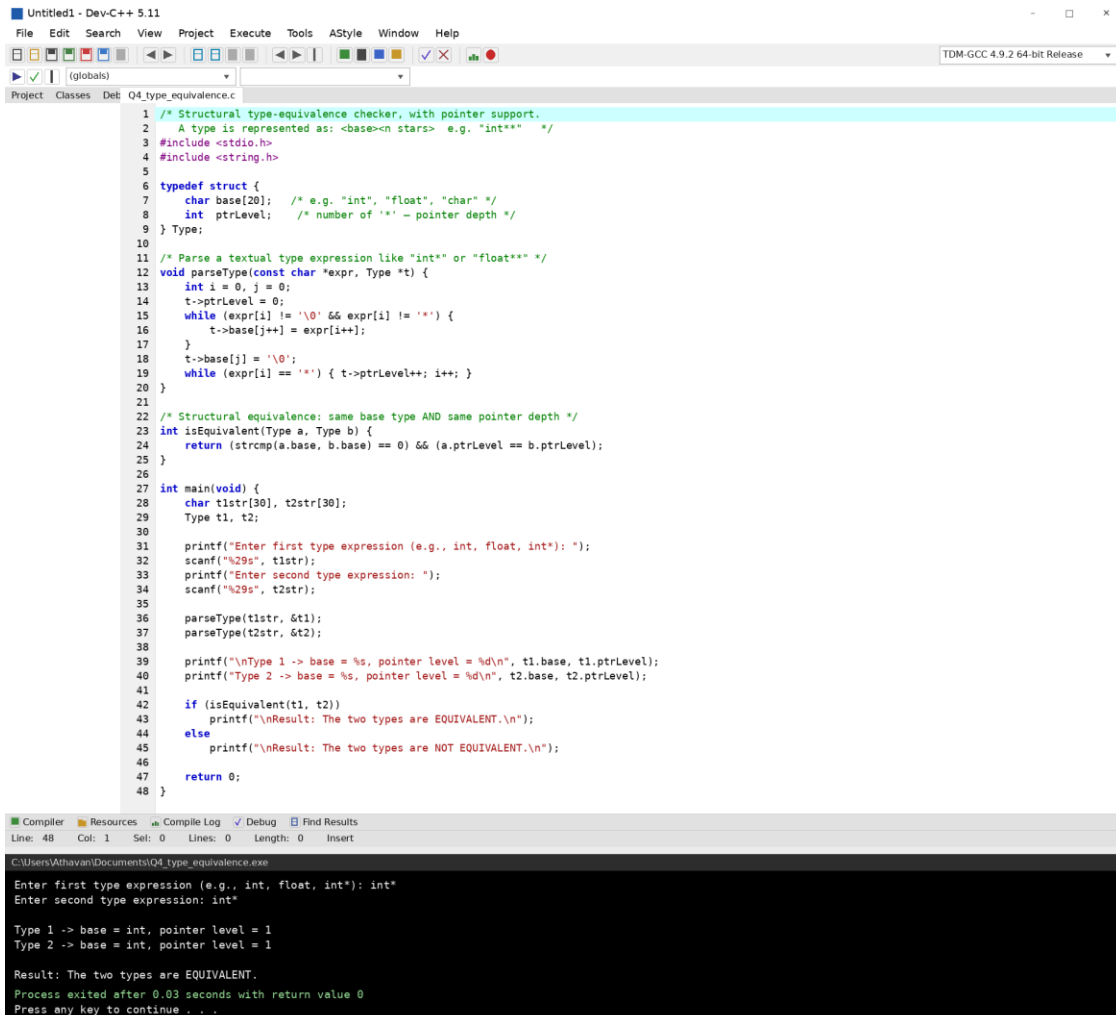
```
1  /* L-attributed evaluator for expressions like a + b * c.
2  The running total is passed DOWN as a function argument,
3  simulating an inherited attribute (E'.inh / T'.inh). */
4  #include <stdio.h>
5  #include <ctype.h>
6
7  const char *src;
8  double vars['z' + 1]; /* simple variable table a..z */
9
10 void skipSpace(void) { while (*src == ' ') src++; }
11
12 double parseFactor(void) {
13     skipSpace();
14     double v;
15     if (isalpha(*src)) {
16         v = vars[(int)*src];
17         src++;
18     } else {
19         v = 0;
20         while (isdigit(*src)) v = v * 10 + (*src++ - '0');
21     }
22     return v;
23 }
24
25 /* T' -> * F T' { inherited value 'inh' flows down } */
26 double parseTermPrime(double inh) {
27     skipSpace();
28     if (*src == '*') {
29         src++;
30         double f = parseFactor();
31         return parseTermPrime(inh * f); /* pass new inh down */
32     }
33     return inh; /* T'.val = T'.inh (epsilon production) */
34 }
35
36 double parseTerm(void) {
37     double f = parseFactor();
38     return parseTermPrime(f); /* T.val = T'.val, seeded with F.val */
39 }
40
41 /* E' -> + T E' { inherited value 'inh' flows down } */
42 double parseExprPrime(double inh) {
43     skipSpace();
44     if (*src == '+') {
45         src++;
46         double t = parseTerm();
47         return parseExprPrime(inh + t); /* pass new inh down */
48     }
49     return inh; /* E'.val = E'.inh (epsilon production) */
50 }
51
52 double parseExpr(void) {
53     double t = parseTerm();
54     return parseExprPrime(t); /* E.val = E'.val, seeded with T.val */
55 }
56
57 int main(void) {
58     vars['a'] = 2; vars['b'] = 3; vars['c'] = 4; /* a=2, b=3, c=4 */
59     char expr[] = "a + b * c";
60     src = expr;
61     double result = parseExpr();
62     printf("Expression: a + b * c (a=2, b=3, c=4)\n");
63     printf("Result = %.2lf\n", result);
64     return 0;
65 }
```

Compiler Resources Compile Log Debug Find Results
Line: 65 Col: 1 Sel: 0 Lines: 0 Length: 0 Insert
C:\Users\Athavan\Documents\Q3_inherited_attributes.exe
Expression: a + b * c (a=2, b=3, c=4)
Result = 14.00
Process exited after 0.03 seconds with return value 0
Press any key to continue . . .

Q4. Type Equivalence Checker (with Pointer Types)

- Accepts two type expressions (e.g., int, float, int*).
- Compares them structurally for type equivalence (base type + pointer depth).
- Displays whether the types are Equivalent or Not Equivalent, extended to handle pointer types.

Program (Dev-C++) and Console Output:



```
1  /* Structural type-equivalence checker, with pointer support.
2  A type is represented as: <base><n stars> e.g. "int*" */
3  #include <stdio.h>
4  #include <string.h>
5
6  typedef struct {
7      char base[20]; /* e.g. "int", "float", "char" */
8      int ptrLevel; /* number of '*' - pointer depth */
9  } Type;
10
11 /* Parse a textual type expression like "int*" or "float*" */
12 void parseType(const char *expr, Type *t) {
13     int i = 0, j = 0;
14     t->ptrLevel = 0;
15     while (expr[i] != '\0' && expr[i] != '*') {
16         t->base[j++] = expr[i++];
17     }
18     t->base[j] = '\0';
19     while (expr[i] == '*') { t->ptrLevel++; i++; }
20 }
21
22 /* Structural equivalence: same base type AND same pointer depth */
23 int isEquivalent(Type a, Type b) {
24     return (strcmp(a.base, b.base) == 0) && (a.ptrLevel == b.ptrLevel);
25 }
26
27 int main(void) {
28     char t1str[30], t2str[30];
29     Type t1, t2;
30
31     printf("Enter first type expression (e.g., int, float, int*): ");
32     scanf("%29s", t1str);
33     printf("Enter second type expression: ");
34     scanf("%29s", t2str);
35
36     parseType(t1str, &t1);
37     parseType(t2str, &t2);
38
39     printf("\nType 1 -> base = %s, pointer level = %d\n", t1.base, t1.ptrLevel);
40     printf("Type 2 -> base = %s, pointer level = %d\n", t2.base, t2.ptrLevel);
41
42     if (isEquivalent(t1, t2))
43         printf("\nResult: The two types are EQUIVALENT.\n");
44     else
45         printf("\nResult: The two types are NOT EQUIVALENT.\n");
46
47     return 0;
48 }
```

Compiler Resources Compile Log Debug Find Results
Line: 48 Col: 1 Sel: 0 Lines: 0 Length: 0 Insert

C:\Users\Athavan\Documents\Q4_type_equivalence.exe

```
Enter first type expression (e.g., int, float, int*): int*
Enter second type expression: int*

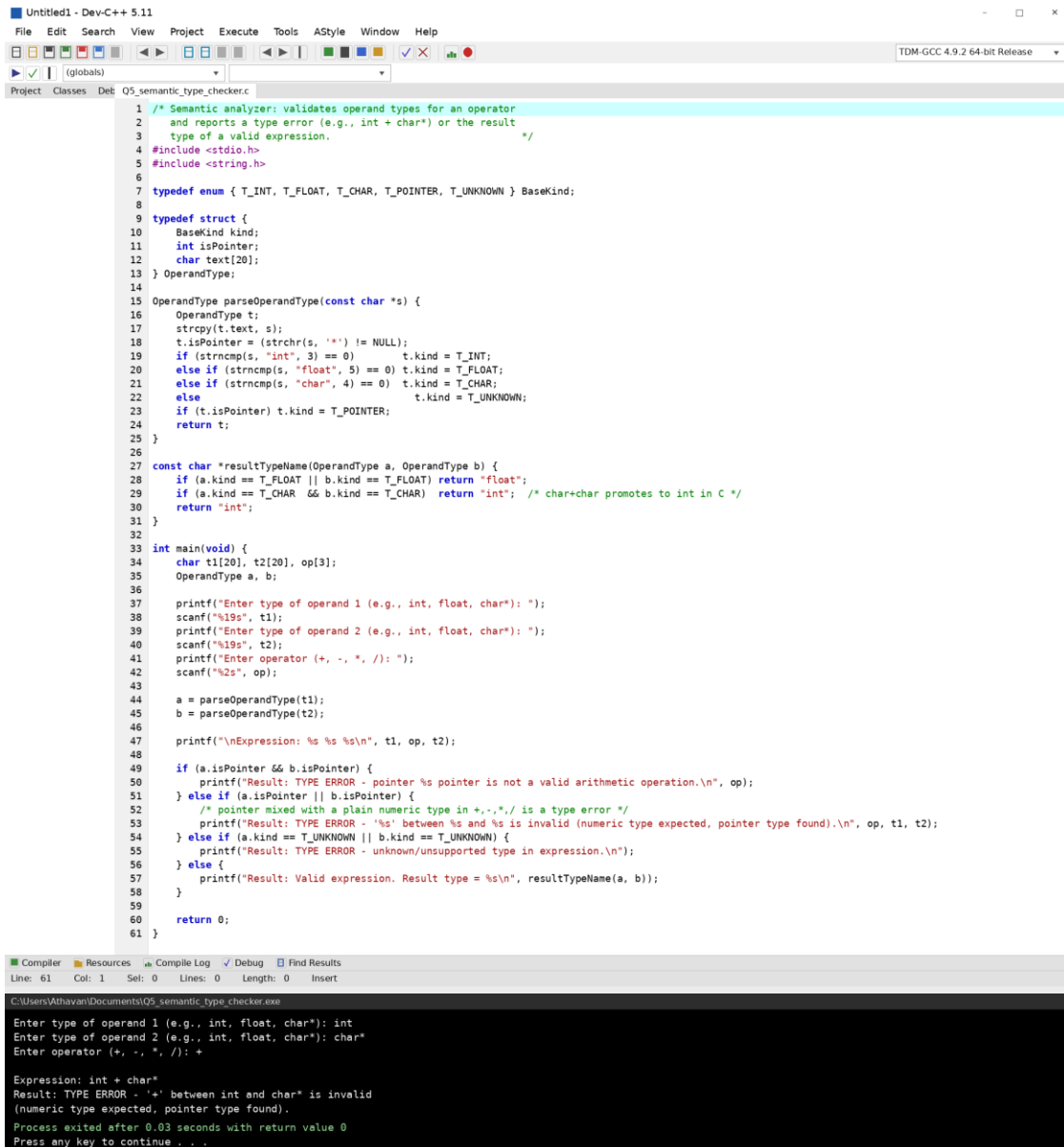
Type 1 -> base = int, pointer level = 1
Type 2 -> base = int, pointer level = 1

Result: The two types are EQUIVALENT.
Process exited after 0.03 seconds with return value 0
Press any key to continue . . .
```

Q5. Semantic Analyzer — Type Checking of Arithmetic Expressions

- Accepts two operands and their types.
- Checks whether the operation (e.g., +, *) is valid between the given types.
- Reports a Valid expression or a Type error (e.g., int + char*), demonstrating semantic-error detection.

Program (Dev-C++) and Console Output:



```
1  /* Semantic analyzer: validates operand types for an operator
2  and reports a type error (e.g., int + char*) or the result
3  type of a valid expression. */
4  #include <stdio.h>
5  #include <string.h>
6
7  typedef enum { T_INT, T_FLOAT, T_CHAR, T_POINTER, T_UNKNOWN } BaseKind;
8
9  typedef struct {
10     BaseKind kind;
11     int isPointer;
12     char text[20];
13 } OperandType;
14
15 OperandType parseOperandType(const char *s) {
16     OperandType t;
17     strcpy(t.text, s);
18     t.isPointer = (strchr(s, '*') != NULL);
19     if (strncmp(s, "int", 3) == 0) t.kind = T_INT;
20     else if (strncmp(s, "float", 5) == 0) t.kind = T_FLOAT;
21     else if (strncmp(s, "char", 4) == 0) t.kind = T_CHAR;
22     else t.kind = T_UNKNOWN;
23     if (t.isPointer) t.kind = T_POINTER;
24     return t;
25 }
26
27 const char *resultTypeName(OperandType a, OperandType b) {
28     if (a.kind == T_FLOAT || b.kind == T_FLOAT) return "float";
29     if (a.kind == T_CHAR && b.kind == T_CHAR) return "int"; /* char+char promotes to int in C */
30     return "int";
31 }
32
33 int main(void) {
34     char t1[20], t2[20], op[3];
35     OperandType a, b;
36
37     printf("Enter type of operand 1 (e.g., int, float, char*): ");
38     scanf("%19s", t1);
39     printf("Enter type of operand 2 (e.g., int, float, char*): ");
40     scanf("%19s", t2);
41     printf("Enter operator (+, -, *, /): ");
42     scanf("%2s", op);
43
44     a = parseOperandType(t1);
45     b = parseOperandType(t2);
46
47     printf("\nExpression: %s %s %s\n", t1, op, t2);
48
49     if (a.isPointer && b.isPointer) {
50         printf("Result: TYPE ERROR - pointer %s pointer is not a valid arithmetic operation.\n", op);
51     } else if (a.isPointer || b.isPointer) {
52         /* pointer mixed with a plain numeric type in +, -, *, / is a type error */
53         printf("Result: TYPE ERROR - '%s' between %s and %s is invalid (numeric type expected, pointer type found).\n", op, t1, t2);
54     } else if (a.kind == T_UNKNOWN || b.kind == T_UNKNOWN) {
55         printf("Result: TYPE ERROR - unknown/unsupported type in expression.\n");
56     } else {
57         printf("Result: Valid expression. Result type = %s\n", resultTypeName(a, b));
58     }
59
60     return 0;
61 }
```

```
C:\Users\Athavan\Documents\Q5_semantic_type_checker.exe
Enter type of operand 1 (e.g., int, float, char*): int
Enter type of operand 2 (e.g., int, float, char*): char*
Enter operator (+, -, *, /): +

Expression: int + char*
Result: TYPE ERROR - '+' between int and char* is invalid
(numeric type expected, pointer type found).
Process exited after 0.03 seconds with return value 0
Press any key to continue . . .
```